



miMicro

Plataforma de Transporte Inteligente — Santa Cruz de la Sierra, Bolivia

Materia: Estructuras de Datos | **Semestre:** 3er semestre 2026

Universidad: UPSA — Universidad Privada de Santa Cruz de la Sierra

Versión: 1.0 Prototipo | **Fecha:** Mayo 2026

Hector Marquez — 2025111291

Bruno Parada — 2025113242

Cristhian Arze — 2025114451

Javier Caye — 2025111797

Roberto Gutierrez — 2025111916

Samuel Carrasco — 2025211490

Índice

- 1. Descripción del Proyecto
- 2. Stack Tecnológico
- 3. Diagrama de Arquitectura del Sistema
- 4. Estructuras de Datos C++ (17 clases .h)
 - 1. CLASES HECTOR — AnalizadorFlujo, ReporteSeguridad
 - 2. CLASES BRUNO — Persona, TarjetaTransporte, GestorPagos
 - 3. CLASES CRISTHIAN — GrafoParadas, OptimizadorRuta, Ruta
 - 4. CLASES JAVIER — GestorGPS, Parada, PilaHistorial
 - 5. CLASES ROBERTO — Micro, ListaPasajeros, SensorPuerta
 - 6. CLASES SAMUEL — ColaEspera, NotificacionAlerta, TiempoEstimado
- 5. Base de Datos PostgreSQL (schema.sql + seed.sql completos)
- 6. Backend API — Módulos y Endpoints

7. Frontend — Pantallas por Rol
8. Diagrama de Flujo del Sistema
9. Variables de Entorno y Configuración
10. Código Fuente de Archivos Clave (main.py, vite.config.js, package.json, setup.py)
11. Guía Completa para Recrear el Proyecto

1. Descripción del Proyecto

El Problema

El transporte público en Santa Cruz de la Sierra opera de manera informal, generando estas problemáticas para los usuarios:

- **Incertidumbre total:** No se sabe cuándo llegará el siguiente micro ni si tendrá lugar disponible.
- **Cambios de ruta sin aviso:** Los choferes alteran recorridos por tráfico o decisión propia.
- **Pagos ineficientes:** Solo efectivo, genera conflictos por cambio y riesgos de robo.
- **Vacío tecnológico:** Aplicaciones globales como Google Maps no cubren las rutas informales locales.

La Solución — miMicro

miMicro es una plataforma web con tres actores principales que se complementan:

**Pasajero**

Ve el mapa en tiempo real con la posición de los micros, consulta tiempos de llegada (ETA), gestiona su tarjeta de transporte digital y recibe notificaciones de desvíos.

**Chofer**

Transmite su posición GPS en tiempo real, reporta el nivel de ocupación del micro y envía alertas de desvíos o incidentes a los pasajeros de su ruta.

**Administrador**

Gestiona usuarios (cambio de roles), administra rutas y micros de la flota, y consulta estadísticas y logs de seguridad del sistema.

Objetivos académicos

El proyecto aplica las siguientes estructuras de datos estudiadas en la materia:

Estructura	Clase implementada	Uso en el sistema
Pila (Stack)	PilaHistorial	Historial de viajes LIFO por usuario
Cola (Queue)	ColaEspera	Cola de pasajeros esperando en parada
Grafo	GrafoParadas	Red de paradas de una ruta
Lista Enlazada	ListaPasajeros	Pasajeros activos a bordo de un micro
Árbol de prioridad (min-heap)	OptimizadorRuta	Dijkstra para camino más corto
Map (árbol AVL interno)	AnalizadorFlujo	Indexación de datos de flujo por ruta/hora

2. Stack Tecnológico

Backend

Tecnología	Versión	Función
Python	3.11	Lenguaje principal del servidor
FastAPI	0.111	Framework REST API asíncrono
Uvicorn	0.29	Servidor ASGI de producción
python-socketio	5.11	WebSockets para GPS en tiempo real
asyncpg	0.29	Driver PostgreSQL asíncrono (sin ORM)
redis-py	5.0	Caché de posiciones GPS (TTL 30s)
python-jose	3.3	Generación y verificación de JWT
bcrypt	4.2	Hash de contraseñas (bcrypt rounds=12)
pybind11	2.12	Bindings Python↔C++ para las estructuras
python-dotenv	1.0	Carga de variables de entorno (.env)

Frontend

Tecnología	Versión	Función
React	18.3	Biblioteca UI
Vite	5.4	Build tool y dev server
React Router	v6	Navegación client-side
socket.io-client	4.7	WebSocket cliente para GPS
react-leaflet	4.2	Mapas interactivos (OpenStreetMap)
leaflet	1.9	Motor de mapas base

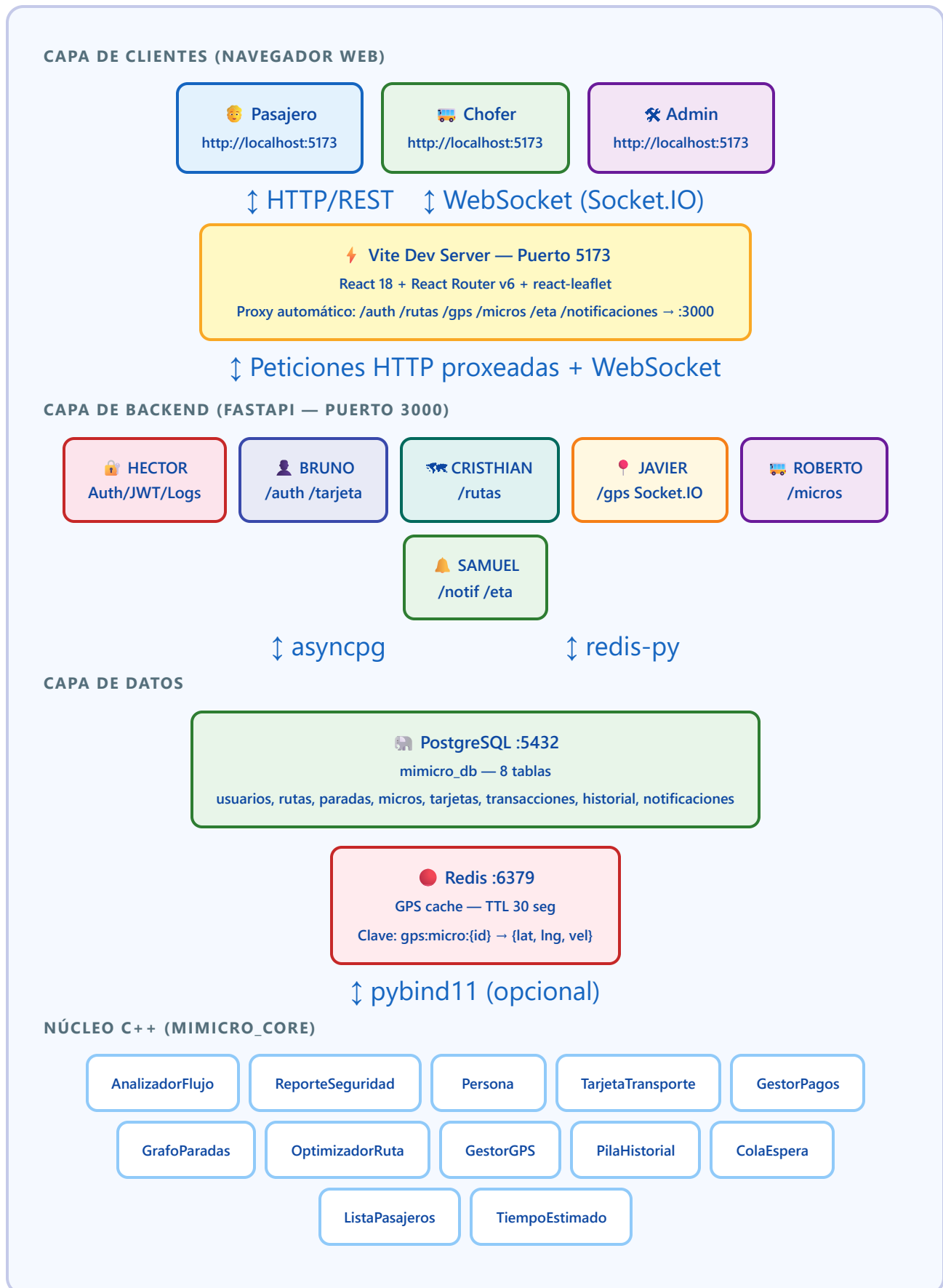
Base de datos e infraestructura

Tecnología	Función
PostgreSQL 14+	Base de datos relacional principal
Redis 7+	Caché en memoria para posiciones GPS

C++ Core

Las 18 clases C++ están implementadas como `.h` headers only (sin archivos `.cpp` separados). Se vinculan al backend Python mediante **pybind11**. Si no se compilan, el backend usa implementaciones de fallback en Python puro.

3. Diagrama de Arquitectura del Sistema



Flujo de comunicación GPS (tiempo real)

- 1. Chofer emite:** `navigator.geolocation.watchPosition()` → frontend llama `emitirGPS(microId, lat, lng, vel)` → Socket.IO evento `gps_update`
- 2. Backend recibe:** Escribe en Redis `gps:micro:{id}` con TTL 30s → emite `micro_moved` a sala `ruta:{rutaId}`
- 3. Pasajero recibe:** Suscripto via `join_ruta` → escucha `micro_moved` → actualiza marcador en el mapa
- 4. Al parar:** Chofer emite `gps_stop` → backend borra Redis key → emite `micro_offline` → mapa quita el marcador

4. Estructuras de Datos C++

Todas las clases están definidas exclusivamente en archivos `.h`. El lenguaje usado (nombres de variables, comentarios, métodos) es **español**.

4.1 CLASES HECTOR — Seguridad y Análisis de Flujo

AnalizadorFlujo `std::map<string, DatoFlujo>`

Analiza el tráfico de pasajeros por ruta y hora del día. Usa un **mapa indexado** con clave compuesta `"rutaId_hora"` para acceso $O(\log n)$.

Atributo / Método	Tipo	Descripción
DatoFlujo::rutald	int	ID de la ruta
DatoFlujo::hora	int	Hora del día (0–23)
DatoFlujo::cantidadViajes	int	Número de viajes registrados
DatoFlujo::totalPasajeros	int	Total de pasajeros en esa franja
registrarViaje(rutald, hora, pasajeros)	void	Acumula datos de un viaje
obtenerFlujo(rutald, hora)	DatoFlujo	Consulta datos específicos
getTotalViajes()	int	Suma todos los viajes registrados
getMicrosActivos() / setMicrosActivos(n)	int/void	Contador de micros en servicio

ReporteSeguridad `std::vector<LogSeguridad>`

Sistema de logging de eventos de seguridad. En Python se serializa a archivo JSONL (`security.log`).

Atributo / Método	Tipo	Descripción
LogSeguridad::eventold	int	ID autoincremental
LogSeguridad::tipoEvento	string	LOGIN_OK / LOGIN_FAIL / ERROR / WARNING
LogSeguridad::usuario	string	Email del usuario
LogSeguridad::ip	string	IP de origen
LogSeguridad::timestamp	string	ISO 8601 UTC
LogSeguridad::nivel	string	INFO / WARNING / ERROR
registrarAcceso(usuario, ip, exitoso)	void	Log de intento de login

registrarError(evento, detalle)	void	Log de error del sistema
obtenerLogs(n)	vector	Últimos N registros
obtenerLogsPorNivel(nivel)	vector	Filtrar por severidad

4.2 CLASES BRUNO — Usuarios y Pagos

Persona

Representa a un usuario del sistema. Campos públicos para acceso directo desde pybind11.

Atributo / Método	Tipo	Descripción
id, nombre, email	int/string	Identificación del usuario
passwordHash	string	Hash bcrypt de la contraseña
rol	string	pasajero / chofer / admin / suspendido
telefono	string	Número de contacto (opcional)
esAdmin() / esChofer() / esPasajero()	bool	Verificación de rol
toString()	string	Representación legible

TarjetaTransporte

Tarjeta de pago digital con saldo. Cada pasajero tiene exactamente una tarjeta.

Atributo / Método	Tipo	Descripción
id, usuariold	int	Identificadores
saldo	double	Saldo disponible en Bolivianos
numeroTarjeta	string	Ej: TC-000001
activa	bool	Estado de la tarjeta
recargar(monto)	bool	Suma saldo (monto > 0)
cobrar(monto)	bool	Descuenta saldo (verifica fondos)
activar() / desactivar()	void	Cambia estado

GestorPagos

Administra múltiples tarjetas y el historial de transacciones.

Atributo / Método	Tipo	Descripción
Transaccion::id, tarjetald, monto	int/double	Datos de la transacción
Transaccion::tipo	string	recarga / cobro
agregarTarjeta(tarjeta)	void	Registra tarjeta en el gestor
recargarTarjeta(id, monto)	bool	Recarga con validación
cobrarPasaje(id, monto)	bool	Cobro de tarifa
consultarSaldo(id)	double	Saldo actual
obtenerHistorial(id)	vector<Transaccion>	Historial de movimientos

4.3 CLASES CRISTHIAN — Rutas y Optimización

GrafoParadas Lista de adyacencia — std::map

Grafo bidireccional donde cada nodo es una parada y cada arista tiene un peso (distancia en km calculada con Haversine).

Atributo / Método	Tipo	Descripción
AristaGrafo::destino	int	ID de la parada destino
AristaGrafo::peso	double	Distancia en km
agregarParada(id)	void	Agrega nodo al grafo
agregarArista(origen, destino, peso)	void	Arista bidireccional
obtenerVecinos(id)	vector<AristaGrafo>	Paradas adyacentes
existeParada(id)	bool	Verificación $O(\log n)$
obtenerTodasParadas()	vector<int>	Todos los nodos
getNumParadas()	int	Total de nodos

OptimizadorRuta Dijkstra — std::priority_queue (min-heap)

Implementación del algoritmo de Dijkstra para encontrar el camino más corto entre dos paradas de una ruta.

Atributo / Método	Tipo	Descripción
ResultadoRuta::camino	vector<int>	Secuencia de IDs de paradas
ResultadoRuta::distanciaTotal	double	Distancia total en km
ResultadoRuta::encontrado	bool	Indica si existe camino
dijkstra(grafo, origen, destino)	ResultadoRuta	Algoritmo principal $O((V+E)\cdot\log V)$
encontrarRutaOptima(grafo, orig, dest)	ResultadoRuta	Wrapper con validación

```
// Pseudocódigo Dijkstra implementado
priority_queue<pair<double,int>, vector, greater> pq;
map<int,double> distancias; // inicializadas a infinito
map<int,int> previos; // para reconstruir el camino
pq.push({0.0, origen});
distancias[origen] = 0.0;
while (!pq.empty()) {
    auto [dist, u] = pq.top(); pq.pop();
    for (auto& arista : grafo.obtenerVecinos(u)) {
        double nueva = dist + arista.peso;
        if (nueva < distancias[arista.destino]) {
            distancias[arista.destino] = nueva;
            previos[arista.destino] = u;
            pq.push({nueva, arista.destino});
        }
    }
}
```

Ruta

Atributo / Método	Tipo	Descripción
id, nombre, descripcion	int/string	Datos básicos de la ruta
activa	bool	Estado de operación
paradaIds	vector<int>	IDs de paradas en orden
agregarParada(id)	void	Añade parada al final
tieneParada(id)	bool	Verifica membresía
desactivar()	void	Marca inactiva

4.4 CLASES JAVIER — GPS e Historial

GestorGPS `std::map<int, PosicionGPS>`

Mantiene el registro en memoria de las posiciones activas. En producción es reemplazado por Redis con TTL de 30 segundos.

Atributo / Método	Tipo	Descripción
PosicionGPS::microId, lat, lng	int/double	Identificación y coordenadas
PosicionGPS::timestamp	string	Momento del registro
PosicionGPS::velocidad	double	Km/h
actualizarPosicion(microId, lat, lng, vel)	void	Inserta o actualiza
obtenerPosicion(microId)	PosicionGPS	Última posición conocida
microActivo(microId)	bool	Verifica si tiene posición activa
obtenerTodas()	vector	Todas las posiciones activas
eliminarMicro(microId)	void	Quita del tracking (fin de ruta)

Parada

Atributo / Método	Tipo	Descripción
id, nombre	int/string	Identificación
lat, lng	double	Coordenadas geográficas WGS84
rutalId	int	Ruta a la que pertenece
ordenEnRuta	int	Posición secuencial (1, 2, 3...)
distanciaA(otraParada)	double	Distancia Haversine en km

PilaHistorial std::stack<RegistroViaje>

Pila LIFO que almacena el historial de viajes de un usuario. Capacidad máxima configurable (default: 100 entradas).

Atributo / Método	Tipo	Descripción
RegistroViaje::usuarioId, microId	int	Identificadores del viaje
RegistroViaje::paradaOrigenId, paradaDestinoId	int	Paradas del trayecto
RegistroViaje::fecha	string	Timestamp del viaje
RegistroViaje::costo	double	Costo cobrado en Bs
agregarViaje(registro)	void	Push — elimina el más viejo si llena

obtenerUltimo()	RegistroViaje	Top sin pop
obtenerTodos()	vector	Convierte pila a vector
estaVacia()	bool	Verificación
limpiar()	void	Vacía la pila

4.5 CLASES ROBERTO — Micros y Pasajeros

Micro

Atributo / Método	Tipo	Descripción
id, placa	int/string	Identificación del vehículo
choferId, rutaId	int	Relaciones con chofer y ruta
capacidad	int	Capacidad máxima de pasajeros
estado	string	activo / inactivo / mantenimiento
pasajerosAbordo	int	Contador actual
estaActivo()	bool	estado == "activo"
estaLleno()	bool	pasajerosAbordo >= capacidad
getOcupacion()	string	vacio / medio / lleno (por porcentaje)
subirPasajero() / bajarPasajero()	bool	Modifica contador (valida límites)
activar() / desactivar()	void	Cambia estado

ListaPasajeros Lista enlazada simple — NodoPasajero*

Lista enlazada simple donde cada nodo representa un pasajero actualmente a bordo de un micro.

Atributo / Método	Tipo	Descripción
NodoPasajero::usuarioid	int	ID del pasajero
NodoPasajero::nombre	string	Nombre del pasajero
NodoPasajero::paradaOrigen	int	Parada donde abordó
NodoPasajero::siguiente	NodoPasajero*	Puntero al siguiente nodo

agregarPasajero(id, nombre, parada)	void	Inserta al frente O(1)
eliminarPasajero(usuariold)	bool	Búsqueda y eliminación O(n)
existePasajero(usuariold)	bool	Búsqueda O(n)
obtenerIds()	vector<int>	Lista de todos los IDs
getTamanio()	int	Recorre la lista O(n)
~ListaPasajeros()	destructor	Libera toda la memoria

SensorPuerta

Simula un sensor físico de la puerta del micro. Cuenta entradas y salidas para determinar ocupación real.

Atributo / Método	Tipo	Descripción
puertaAbierta	bool	Estado actual de la puerta
entradas, salidas	int	Contadores de movimiento
abrirPuerta() / cerrarPuerta()	void	Cambio de estado
registrarEntrada() / registrarSalida()	void	Incrementa contadores
getPasajerosNetos()	int	entradas - salidas
resetear()	void	Pone contadores a 0

4.6 CLASES SAMUEL — Cola, ETA y Notificaciones

ColaEspera `std::queue<PasajeroEspera>`

Cola FIFO de pasajeros esperando en una parada específica. Cada parada tiene su propia cola.

Atributo / Método	Tipo	Descripción
PasajeroEspera::usuariold	int	ID del pasajero
PasajeroEspera::paradald	int	Parada donde espera
PasajeroEspera::timestamp	string	Hora de llegada a la parada
PasajeroEspera::rutald	int	Ruta de interés del pasajero
agregarPasajero(pasajero)	void	Enqueue O(1)
atenderPasajero()	PasajeroEspera	Dequeue O(1)

estaVacia()	bool	Verificación
getTamanio()	int	Cantidad esperando
getParadaId()	int	Parada asociada a esta cola

NotificacionAlerta `std::vector<Notificacion>`

Atributo / Método	Tipo	Descripción
Notificacion::id, usuariold	int	Identificadores
Notificacion::mensaje	string	Texto de la notificación
Notificacion::tipo	string	desvio / retraso / info / sistema
Notificacion::leida	bool	Estado de lectura
Notificacion::timestamp	string	Fecha/hora de creación
crearNotificacion(usuariold, msg, tipo)	void	Agrega nueva notificación
obtenerParaUsuario(usuariold)	vector	Notificaciones del usuario
marcarLeida(id)	void	Marca como leída
contarNoLeidas(usuariold)	int	Conteo de no leídas

TiempoEstimado

Calcula el ETA de un micro a una parada usando la fórmula de Haversine y una velocidad promedio de 25 km/h.

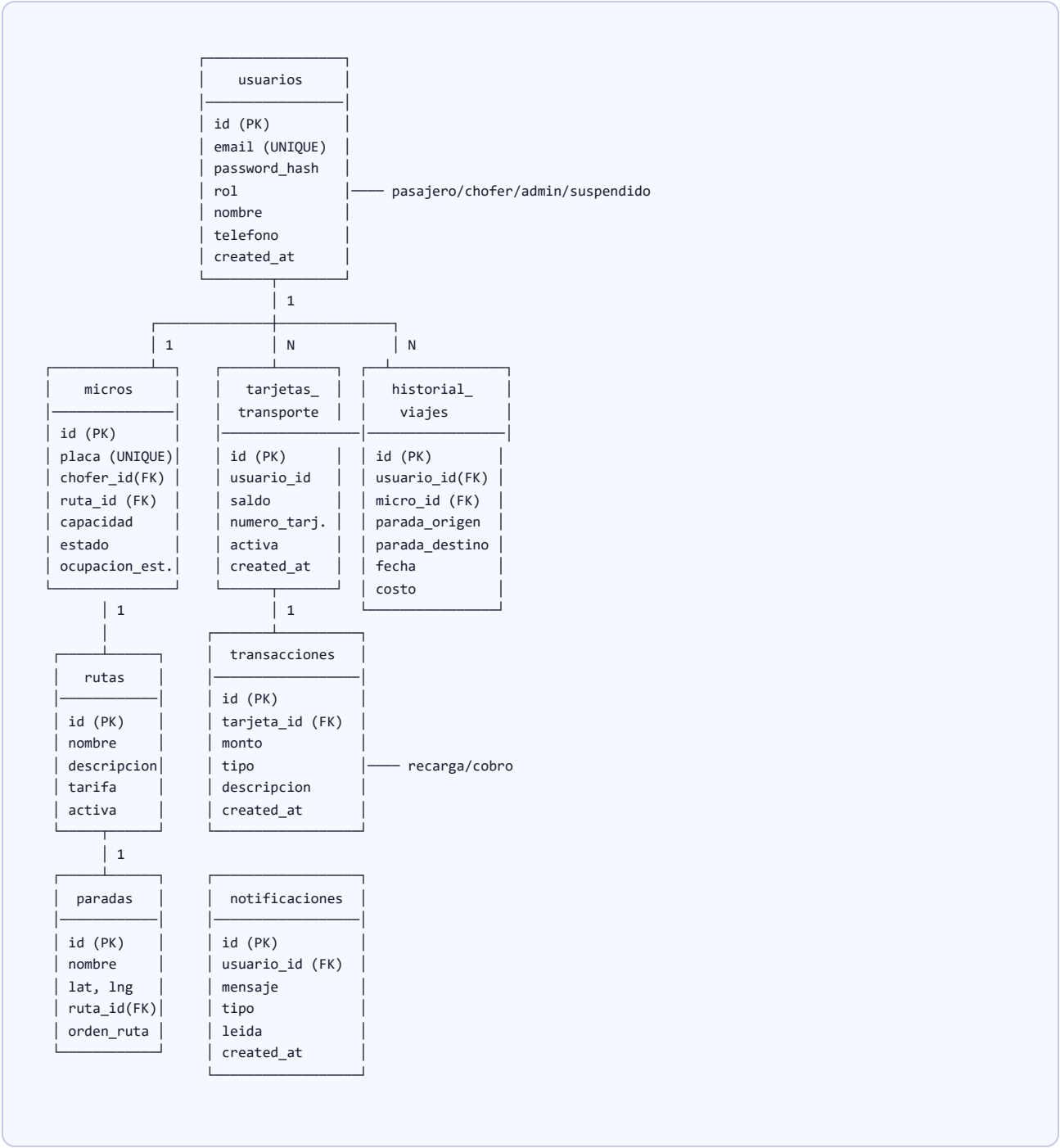
Atributo / Método	Tipo	Descripción
ResultadoETA::microid, paradaId	int	Identificadores
ResultadoETA::distanciaMetros	double	Distancia calculada
ResultadoETA::tiempoSegundos	int	Tiempo estimado de llegada
ResultadoETA::ocupacion	string	Estado de ocupación del micro
haversine(lat1, lng1, lat2, lng2)	double	Distancia en km entre dos puntos
calcularETA(posicion, parada)	ResultadoETA	Cálculo principal
distanciaASegundos(km)	int	km / 25km/h → segundos

```
// Fórmula Haversine implementada
R = 6371.0 // radio Tierra en km
dLat = (lat2 - lat1) * π / 180
dLng = (lng2 - lng1) * π / 180
a = sin(dLat/2)² + cos(lat1) * cos(lat2) * sin(dLng/2)²
c = 2 * atan2(√a, √(1-a))
distancia = R * c
```

5. Base de Datos PostgreSQL

Motor: **PostgreSQL 14+**. Nombre: **mimicro_db**. Acceso: **asyncpg** (driver async sin ORM). 8 tablas con índices para consultas frecuentes.

Diagrama Entidad-Relación (simplificado)



Índices

Índice	Tabla	Columnas	Motivo
idx_micros_ruta	micros	ruta_id	Buscar micros por ruta

idx_paradas_ruta	paradas	ruta_id, orden_en_ruta	Listar paradas en orden
idx_historial_usuario	historial_viajes	usuario_id, fecha DESC	Historial reciente de un usuario
idx_notif_usuario	notificaciones	usuario_id, leida	Notificaciones no leídas
idx_transac_tarjeta	transacciones	tarjeta_id, created_at DESC	Historial de movimientos

Caché Redis

Clave	Valor (JSON)	TTL	Propósito
gps:micro: {id}	{ "lat":..., "lng":..., "velocidad":..., "timestamp":... }	30 seg	Posición actual del micro. Expira si no hay actualización.

Código SQL completo — schema.sql

Ejecutar con: `psql -U postgres -d mimicro_db -f mimicro-app/backend/database/schema.sql`

```
-- miMicro - Esquema PostgreSQL completo

CREATE TABLE IF NOT EXISTS usuarios (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  rol         VARCHAR(20) NOT NULL CHECK (rol IN ('pasajero','chofer','admin','suspendido')),
  nombre      VARCHAR(255) NOT NULL,
  telefono    VARCHAR(20)  DEFAULT '',
  created_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS tarjetas_transporte (
  id          SERIAL PRIMARY KEY,
  usuario_id  INTEGER      REFERENCES usuarios(id) ON DELETE CASCADE,
  saldo       DECIMAL(10,2) DEFAULT 0.00,
  numero_tarjeta VARCHAR(20) UNIQUE NOT NULL,
  activa      BOOLEAN      DEFAULT TRUE,
  created_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS transacciones (
  id          SERIAL PRIMARY KEY,
  tarjeta_id  INTEGER      REFERENCES tarjetas_transporte(id) ON DELETE CASCADE,
  monto       DECIMAL(10,2) NOT NULL,
  tipo        VARCHAR(20)  CHECK (tipo IN ('recarga','cobro')),
  descripcion VARCHAR(255) DEFAULT '',
  created_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS rutas (
  id          SERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  descripcion TEXT          DEFAULT '',
  activa      BOOLEAN      DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS paradas (
  id          SERIAL PRIMARY KEY,
  nombre      VARCHAR(255) NOT NULL,
  lat         DECIMAL(10,8) NOT NULL,
  lng         DECIMAL(11,8) NOT NULL,
  ruta_id     INTEGER      REFERENCES rutas(id) ON DELETE CASCADE,
  orden_en_ruta INTEGER    NOT NULL
);
```

```

CREATE TABLE IF NOT EXISTS micros (
  id          SERIAL PRIMARY KEY,
  placa       VARCHAR(20) UNIQUE NOT NULL,
  chofer_id   INTEGER REFERENCES usuarios(id),
  ruta_id     INTEGER REFERENCES rutas(id),
  capacidad   INTEGER DEFAULT 30,
  estado      VARCHAR(20) DEFAULT 'inactivo' CHECK (estado IN ('activo','inactivo')),
  ocupacion_estado VARCHAR(20) DEFAULT 'vacio' CHECK (ocupacion_estado IN ('vacio','medio','lleno'))
);

CREATE TABLE IF NOT EXISTS historial_viajes (
  id          SERIAL PRIMARY KEY,
  usuario_id  INTEGER REFERENCES usuarios(id),
  micro_id    INTEGER REFERENCES micros(id),
  parada_origen_id INTEGER REFERENCES paradas(id),
  parada_destino_id INTEGER REFERENCES paradas(id),
  fecha       TIMESTAMP DEFAULT NOW(),
  costo       DECIMAL(10,2) DEFAULT 3.50
);

CREATE TABLE IF NOT EXISTS notificaciones (
  id          SERIAL PRIMARY KEY,
  usuario_id  INTEGER REFERENCES usuarios(id) ON DELETE CASCADE,
  mensaje     TEXT NOT NULL,
  tipo        VARCHAR(50) DEFAULT 'info',
  leida       BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Índices para consultas frecuentes
CREATE INDEX IF NOT EXISTS idx_micros_ruta      ON micros(ruta_id);
CREATE INDEX IF NOT EXISTS idx_paradas_ruta     ON paradas(ruta_id, orden_en_ruta);
CREATE INDEX IF NOT EXISTS idx_historial_usuario ON historial_viajes(usuario_id, fecha DESC);
CREATE INDEX IF NOT EXISTS idx_notif_usuario   ON notificaciones(usuario_id, leida);
CREATE INDEX IF NOT EXISTS idx_transacc_tarjeta ON transacciones(tarjeta_id, created_at DESC);

```

Código SQL completo — seed.sql

Ejecutar con: `psql -U postgres -d mimicro_db -f mimicro-app/backend/database/seed.sql`

```

-- miMicro - Datos iniciales para desarrollo/testing
-- IMPORTANTE: Cambiar la contraseña del admin antes de producción.

-- Admin (contraseña: Admin2026!)
INSERT INTO usuarios (email, password_hash, rol, nombre, telefono)
VALUES (
  'admin@mimicro.bo',
  '$2b$12$ouIgyE7fy1RXmjN207x8.QZ7p4IXCvLo7erk6HSF67a96aZ124jW',
  'admin',
  'Administrador miMicro',
  '+591 70000000'
) ON CONFLICT (email) DO NOTHING;

-- Choferes de prueba (contraseña: Chofer123!)
INSERT INTO usuarios (email, password_hash, rol, nombre, telefono) VALUES
('pedro.rios@mimicro.bo', '$2b$12$wKZoHSnZ808S1EVApkeDfE1l1WChvq/6u5S0dYgijofLsrCLcGdUK', 'chofer', 'Pedro Rios', '+591 711111111'),
('carlos.vaca@mimicro.bo', '$2b$12$wKZoHSnZ808S1EVApkeDfE1l1WChvq/6u5S0dYgijofLsrCLcGdUK', 'chofer', 'Carlos Vaca', '+591 722222222')
ON CONFLICT (email) DO NOTHING;

-- Pasajeros de prueba (contraseña: Pasajero123!)
INSERT INTO usuarios (email, password_hash, rol, nombre, telefono) VALUES
('juan.perez@mail.com', '$2b$12$qpaxt0jGZA36z/1o9i7A.es3hw35pbBMQ2etSXh0kucYHA7dA0522', 'pasajero', 'Juan Perez', '+591 765432109'),
('maria.lopez@mail.com', '$2b$12$qpaxt0jGZA36z/1o9i7A.es3hw35pbBMQ2etSXh0kucYHA7dA0522', 'pasajero', 'Maria Lopez', '+591 765432109')
ON CONFLICT (email) DO NOTHING;

-- Tarjetas para los pasajeros (saldo inicial: Bs 50)
INSERT INTO tarjetas_transporte (usuario_id, saldo, numero_tarjeta)
SELECT id, 50.00, 'TC-' || LPAD(id::TEXT, 6, '0')

```

```
FROM usuarios WHERE rol='pasajero'
ON CONFLICT DO NOTHING;

-- Rutas de Santa Cruz de la Sierra
INSERT INTO rutas (nombre, descripcion) VALUES
('Ruta 17 - Terminal / Urbarí',      'Terminal Bimodal → 2do Anillo → Equipetrol → Urbarí'),
('Ruta 24 - Mercado Los Pozos / UV', 'Mercado Los Pozos → Av. Cañoto → UV'),
('Ruta 31 - Plan 3000 / Centro',     'Plan 3000 → Av. Brasil → Plaza 24 de Septiembre')
ON CONFLICT DO NOTHING;

-- Paradas Ruta 17 (ruta_id = 1)
INSERT INTO paradas (nombre, lat, lng, ruta_id, orden_en_ruta) VALUES
('Terminal Bimodal',      -17.78330, -63.18200, 1, 1),
('Av. Cañoto / 2do Anillo', -17.78500, -63.18000, 1, 2),
('Equipetrol Norte',      -17.77500, -63.19500, 1, 3),
('Plaza del Estudiante',   -17.77200, -63.19800, 1, 4),
('Urbarí',                 -17.76800, -63.20500, 1, 5)
ON CONFLICT DO NOTHING;

-- Paradas Ruta 24 (ruta_id = 2)
INSERT INTO paradas (nombre, lat, lng, ruta_id, orden_en_ruta) VALUES
('Mercado Los Pozos',      -17.79100, -63.17200, 2, 1),
('Av. Cañoto Esq. 21 Sept', -17.78600, -63.17900, 2, 2),
('Plaza 24 de Septiembre', -17.78330, -63.18220, 2, 3),
('Universidad UV',         -17.77900, -63.18600, 2, 4)
ON CONFLICT DO NOTHING;

-- Micros (chofer_id se resuelve por email)
INSERT INTO micros (placa, chofer_id, ruta_id, capacidad, estado)
SELECT 'SCZ-1234', u.id, 1, 30, 'inactivo'
FROM usuarios u WHERE u.email='pedro.rios@mimicro.bo'
ON CONFLICT (placa) DO NOTHING;

INSERT INTO micros (placa, chofer_id, ruta_id, capacidad, estado)
SELECT 'SCZ-5678', u.id, 2, 25, 'inactivo'
FROM usuarios u WHERE u.email='carlos.vaca@mimicro.bo'
ON CONFLICT (placa) DO NOTHING;
```

6. Backend API — Módulos y Endpoints

El servidor corre en **http://localhost:3000**. Documentación interactiva disponible en <http://localhost:3000/docs> (Swagger UI generado automáticamente por FastAPI).

Autenticación

Los endpoints protegidos requieren el header: `Authorization: Bearer <token_JWT>`

El token JWT contiene: `{ "sub": userId, "rol": "admin|chofer|pasajero", "exp": timestamp }`

MODULO_BRUNO — /auth y /tarjeta

POST	/auth/login	
Body: {email, password} → {token, rol, nombre}		
POST	/auth/register	
Body: {nombre, email, password, rol, telefono} → {token, rol, nombre}. Solo rol pasajero/chofer.		
GET	/auth/usuarios	Admin
→ {usuarios: [{id, nombre, email, rol, telefono}]}		
PATCH	/auth/usuarios/{id}/rol	Admin
Body: {rol} → {ok, user_id, nuevo_rol}		
GET	/auth/logs	Admin
?limite=50 → {logs: [{timestamp, nivel, evento, usuario, ip}]}		
GET	/tarjeta/{usuario_id}	Auth
→ {id, saldo, numero_tarjeta, activa}		
POST	/tarjeta/{usuario_id}/recargar	Auth
Body: {monto} → {ok, saldo_nuevo}		
POST	/tarjeta/{usuario_id}/cobrar	Auth
Body: {monto, descripcion} → {ok, saldo_nuevo}		
GET	/tarjeta/{usuario_id}/historial	Auth
→ {transacciones: [{id, monto, tipo, descripcion, created_at}]}		

MODULO_CRISTHIAN — /rutas

GET	/rutas	
→ [{id, nombre, descripcion, activa}]		
GET	/rutas/{id}	
→ {id, nombre, descripcion, activa}		
GET	/rutas/{id}/paradas	
→ [{id, nombre, lat, lng, orden_en_ruta}]		
POST	/rutas	Admin
Body: {nombre, descripcion} → {id, nombre, ...}		
PATCH	/rutas/{id}	Admin
Body: {nombre, descripcion} → ruta actualizada		

DELETE

/rutas/{id} Admin

→ {ok: true} — desactivación lógica (activa=false)

GET

/rutas/{id}/optima?origen=X&destino=Y

→ {encontrado, camino:[paradaId,...], distanciaTotal} — Dijkstra C++

MODULO_JAVIER — /gps

GET

/gps/{micro_id}

→ {microId, lat, lng, velocidad, timestamp} o 404 si no activo

GET

/gps/activos

→ [{microId, lat, lng, velocidad, timestamp}] — todas las posiciones en Redis

GET

/gps/historial/{usuario_id} Auth

→ [{id, micro_placa, ruta_nombre, parada_origen, fecha, costo}]

Socket.IO eventos:

`gps_update` {microId, lat, lng, velocidad} → guarda Redis, emite `micro_moved``gps_stop` {microId} → borra Redis, emite `micro_offline``join_ruta` {rutaId} → suscribe a sala ruta:{rutaId}`leave_ruta` {rutaId} → desuscribe

MODULO_ROBERTO — /micros

GET

/micros

→ [{id, placa, ruta_nombre, chofer_nombre, capacidad, estado, ocupacion_estado}]

GET

/micros/{id}

→ micro + cant_pasajeros_abordo

POST

/micros Admin

Body: {placa, capacidad, ruta_id, chofer_id} → micro creado

PATCH

/micros/{id}/ocupacion Chofer

Body: {estado: "vacio"|"medio"|"lleno"} → {ok}

PATCH

/micros/{id}/estado Chofer/Admin

Body: {estado: "activo"|"inactivo"|"mantenimiento"} → {ok}

MODULO_SAMUEL — /notificaciones y /eta

GET

/notificaciones/{usuario_id} Auth

→ [{id, mensaje, tipo, leida, created_at}] — últimas 50

POST

/notificaciones Auth

Body: {usuario_id, mensaje, tipo} → notificación creada

PATCH

/notificaciones/{id}/leida Auth

→ {ok}

PATCH

/notificaciones/todas/{usuario_id}/leidas Auth

→ {ok} — marca todas las notificaciones del usuario como leídas

POST	/notificaciones/desvio Chofer Body: {ruta_id, descripcion} → notifica masivamente a pasajeros de esa ruta (últimas 2h)
GET	/notificaciones/cola/{ruta_id} → [{parada_id, cantidad_esperando}] — estado de colas ColaEspera C++ por parada
GET	/eta/{micro_id}/{parada_id} → {microId, paradaId, distanciaMetros, tiempoSegundos} — ETA de un micro específico
GET	/eta/ruta/{ruta_id}/{parada_id} → [{microId, distanciaMetros, tiempoSegundos, ocupacion}] — ETAs de todos los micros activos en la ruta

7. Frontend — Pantallas por Rol

La interfaz usa **React Router v6** con rutas protegidas por rol. Cada rol tiene su propio conjunto de pantallas y un sidebar de navegación diferente. URL base: **http://localhost:5173**

Pantallas de Autenticación (acceso público)

Login /login

Formulario de inicio de sesión con email y contraseña. Detecta automáticamente el rol del usuario y redirige a su pantalla principal. Incluye enlace a registro.

- Validación de campos en el frontend
- Mensajes de error en tiempo real
- Redirección automática por rol tras autenticación
- Token JWT guardado en localStorage

Registro /register

Formulario de creación de cuenta. Solo permite crear cuentas de **Pasajero** o **Chofer**. Los administradores solo se crean via seed.sql.

- Selector de rol (pasajero / chofer)
- Validación de contraseña (mínimo 6 caracteres)
- Detección de email duplicado (409)
- Login automático tras registro exitoso

Pantallas del Pasajero — 5 pantallas

Acceso desde /pasajero/*. Sidebar azul.

Mapa en Vivo /pasajero/mapa

Pantalla principal del pasajero. Muestra un mapa interactivo centrado en Santa Cruz (lat -17.7833, lng -63.1822) con la posición en tiempo real de todos los micros activos en la ruta seleccionada.

- **Selector de ruta:** pestañas superiores con todas las rutas disponibles
- **Marcadores de paradas:** ícono azul predeterminado de Leaflet en cada parada de la ruta
- **Marcadores de micros:** ícono dorado personalizado, se mueve en tiempo real vía Socket.IO
- **Suscripción automática:** al cambiar de ruta se desuscribe de la anterior y se suscribe a la nueva
- **Evento micro_moved:** actualiza la posición del marcador en el mapa
- **Evento micro_offline:** elimina el marcador del micro del mapa
- **Limpieza:** al desmontar el componente se llama a la función de limpieza del WebSocket

Servicios usados: getRutas(), getParadas(), suscribirMapa(rutald, onMoved, onOffline)

Llegadas (ETA) /pasajero/eta

Muestra el tiempo estimado de llegada de todos los micros activos a la parada seleccionada.

- **Selector de ruta + selector de parada** (dependiente de la ruta)
- **Auto-refresh:** actualiza automáticamente cada 10 segundos
- **Botón refresh manual**
- Por cada micro activo muestra: distancia en metros, tiempo en minutos y segundos, badge de ocupación coloreado
- Colores de ocupación: verde (vacío), amarillo (medio), rojo (lleno)
- Estado vacío si no hay micros activos en la ruta

Servicios usados: getRutas(), getParadas(rutald), getETARuta(rutald, paradald)

Mi Tarjeta /pasajero/tarjeta

Gestión de la tarjeta de transporte digital del pasajero.

- **Tarjeta visual** con número, saldo en Bs, nombre del titular
- **Código QR** generado dinámicamente con el número de tarjeta
- **Modal de recarga:** input de monto con validación (mínimo Bs 5, máximo Bs 500)
- **Historial de transacciones:** últimas 10 operaciones con tipo, monto y fecha
- Colores: recarga en verde, cobro en rojo
- Toast de confirmación/error

Servicios usados: getTarjeta(userId), recargar(userId, monto), getHistorial(userId)

Historial de Viajes /pasajero/historial

Lista completa del historial de viajes del pasajero, mostrando los viajes más recientes primero (implementado con PilaHistorial en C++).

- Tabla con: placa del micro, nombre de la ruta, paradas de origen y destino, fecha/hora, costo en Bs
- Formato de fecha localizado (es-BO)
- Estado de carga con spinner
- Estado vacío si no tiene viajes registrados

Endpoint: GET /gps/historial/{userId}

Notificaciones /pasajero/notificaciones

Centro de notificaciones del pasajero. Recibe alertas de desvíos, retrasos e información del sistema.

- Lista de notificaciones con badge de tipo (desvío=rojo, retraso=amarillo, info=azul)
- Contador de no leídas en el header
- Clic en notificación la marca como leída (feedback visual inmediato)
- Botón "Marcar todas como leídas" (aparece solo si hay no leídas)
- Estado vacío si no hay notificaciones

Servicios usados: getNotificaciones(userId), marcarLeida(id), marcarTodasLeidas(userId)

Pantallas del Chofer — 3 pantallas

Acceso desde /chofer/*. Sidebar verde.

Ruta Activa /chofer/ruta

Panel principal del chofer. Activa/desactiva la transmisión de posición GPS en tiempo real.

- **Botón "Iniciar Ruta":** activa el micro en la base de datos, inicia `navigator.geolocation.watchPosition()`
- **Indicador de estado** (animado en verde cuando activo)

- **Coordenadas en tiempo real:** muestra lat, lng y velocidad actuales
- **Mapa integrado:** muestra la posición del chofer y las paradas de su ruta
- **Emisión GPS:** cada actualización de posición envía evento `gps_update` via Socket.IO
- **Botón "Detener Ruta":** limpia el watchPosition, desactiva el micro, emite `gps_stop`
- Manejo de error si el navegador no tiene geolocalización habilitada

APIs: GET /micros (busca su micro asignado), PATCH /micros/{id}/estado, socket emitirGPS(), detenerGPS()

Reportar Ocupación `/chofer/ocupacion`

Permite al chofer reportar el nivel de ocupación actual de su micro con un solo clic.

- **3 botones grandes** con emoji e instrucciones claras:
 -  **Vacío** — menos del 40% de capacidad
 -  **Medio** — entre 40% y 80%
 -  **Lleno** — más del 80%
- Muestra el último nivel reportado
- Botones deshabilitados durante el envío
- Toast de confirmación

API: PATCH /micros/{id}/ocupacion con body {estado: "vacio"|"medio"|"lleno"}

Reportar Desvío `/chofer/desvio`

Envía una alerta masiva a todos los pasajeros que viajaron recientemente en la ruta afectada.

- **Selector de ruta:** precargado con la ruta del chofer
- **Textarea:** descripción del desvío o incidente
- Validación de campos requeridos
- Al enviar: el backend crea una notificación para cada pasajero reciente de esa ruta
- Toast de éxito al confirmarse el envío
- Limpia el formulario tras éxito

API: POST /notificaciones/desvio con body {ruta_id, descripcion}

Pantallas del Administrador — 4 pantallas

Acceso desde `/admin/*`. Sidebar violeta. Solo accesible con rol **admin**.

Gestión de Usuarios `/admin/usuarios`

Tabla completa de todos los usuarios registrados en el sistema.

- Tabla: ID, nombre, email, teléfono, badge de rol (coloreado)
- **Buscador en tiempo real** por nombre o email (filtrado en el frontend)
- **Selector de rol inline** por cada usuario: al cambiar el select se guarda inmediatamente via API
- Roles disponibles: pasajero, chofer, admin, suspendido
- Badge de conteo de usuarios totales en el header

- Toast de confirmación/error por cada cambio

APIs: GET /auth/usuarios, PATCH /auth/usuarios/{id}/rol

Gestión de Rutas /admin/rutas

Administración del catálogo de rutas de la red de transporte.

- Tabla: ID, nombre, descripción, tarifa en Bs
- **Botón "Nueva ruta":** abre modal con formulario (nombre*, descripción, tarifa)
- La nueva ruta se agrega a la tabla en tiempo real (optimistic update)
- **Botón "Eliminar"** por fila con confirmación via `window.confirm()`
- Estado vacío si no hay rutas

APIs: GET /rutas, POST /rutas, DELETE /rutas/{id}

Gestión de Micros /admin/micros

Administración de la flota de microbuses.

- Tabla: ID, placa, ruta asignada, capacidad, badge de ocupación, badge de estado
- **Botón "Nuevo micro":** abre modal con formulario (placa*, capacidad, ruta*, ID del chofer opcional)
- Placa se normaliza a mayúsculas automáticamente
- Selector de ruta poblado con todas las rutas disponibles
- Estado vacío si no hay micros

APIs: GET /micros, POST /micros, GET /rutas

Reportes y Estadísticas /admin/reportes

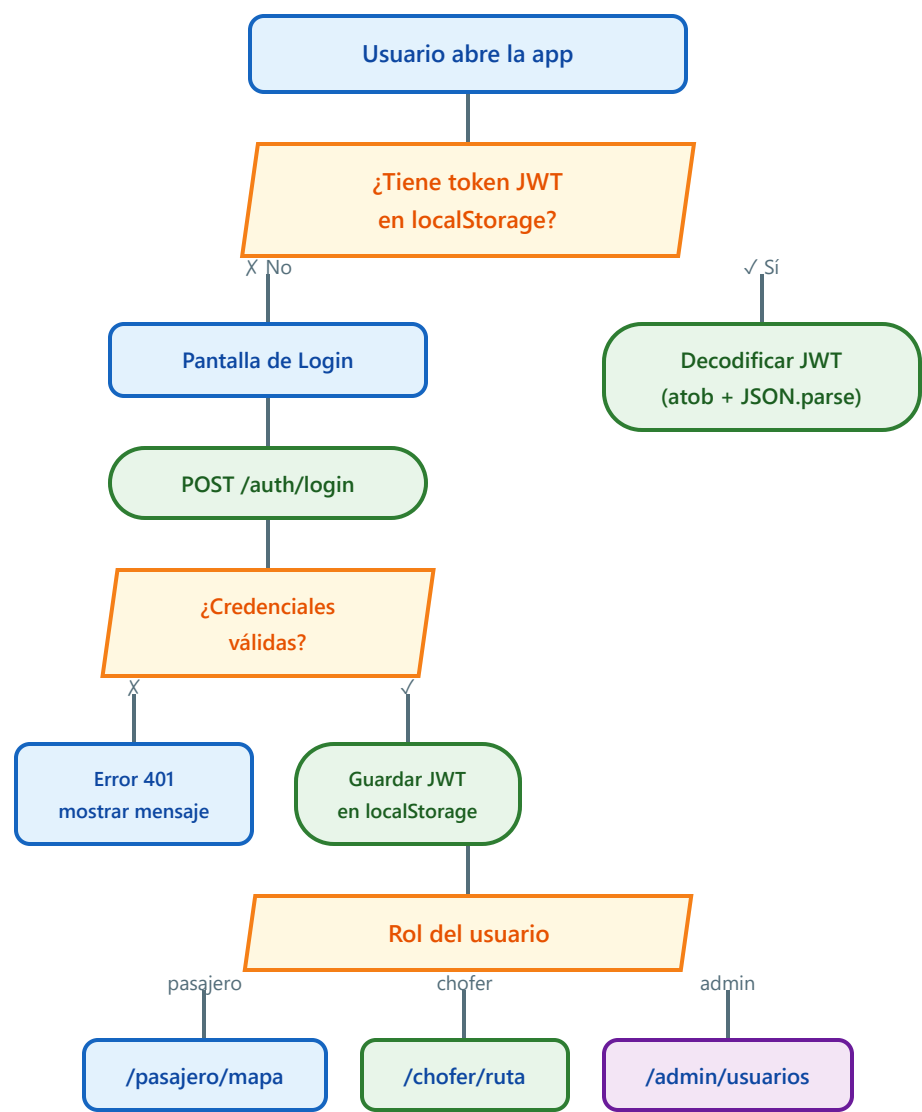
Panel de métricas generales del sistema y log de seguridad.

- **4 tarjetas de estadísticas:**
 - Total usuarios registrados
 - Rutas activas en el sistema
 - Micros en la flota
 - Micros con estado "activo" en este momento
- **Tabla de logs de seguridad:** últimas 50 entradas con timestamp, badge de nivel (INFO/WARNING/ERROR), tipo de evento, usuario e IP
- Los logs se ordenan de más reciente a más antiguo
- Carga todos los datos en paralelo con `Promise.all()`

APIs: GET /auth/usuarios, GET /rutas, GET /micros, GET /auth/logs?limite=50

8. Diagrama de Flujo del Sistema

8.1 Flujo de autenticación y acceso

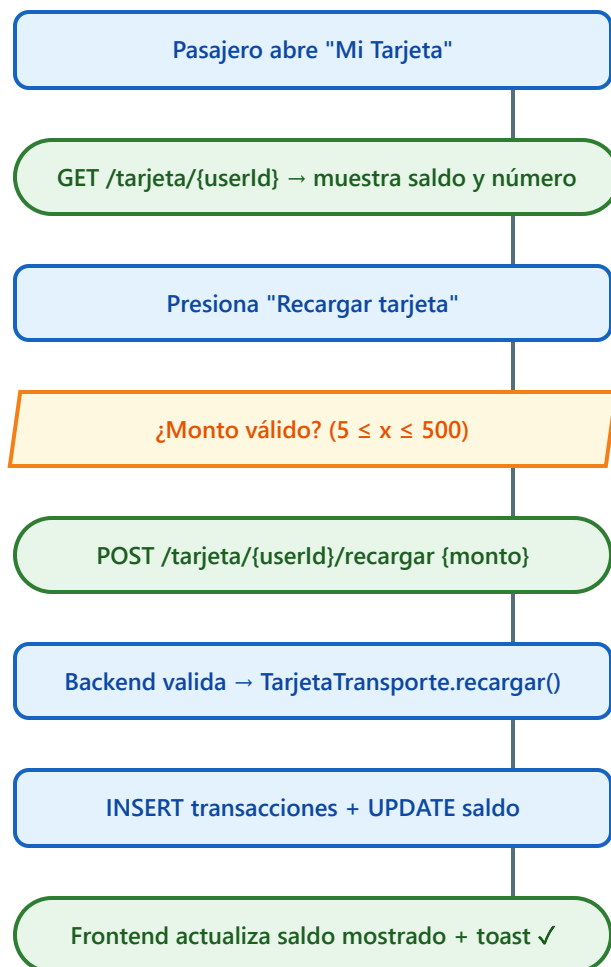


8.2 Flujo GPS en tiempo real (Chofer → Pasajero)

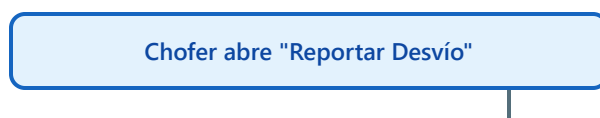
Actor	Paso	Tecnología
 Chofer	Presiona "Iniciar Ruta" en su pantalla	React onClick
 Chofer	Llama PATCH /micros/{id}/estado → {estado: "activo"}	fetch() API
 Chofer	Inicia navigator.geolocation.watchPosition()	Web Geolocation API
 Chofer	Emite evento gps_update {microId, lat, lng, velocidad}	Socket.IO emit
 Backend	Recibe gps_update en gpsSocket.py	python-socketio

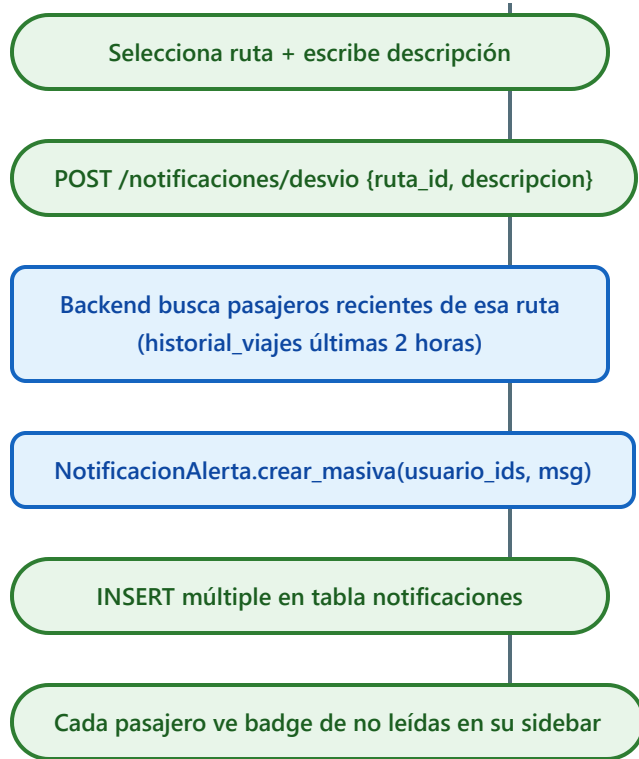
Backend	Guarda en Redis: <code>gps:micro:{id} = {lat, lng, vel}</code> TTL 30s	redis-py SETEX
Backend	Emite <code>micro_moved</code> a sala <code>ruta:{rutaid}</code>	<code>sio.emit(room=)</code>
Pasajero	Recibe evento <code>micro_moved</code> (suscripto a <code>join_ruta</code>)	Socket.IO <code>on()</code>
Pasajero	Actualiza el marcador en el mapa con nuevas coordenadas	react-leaflet state
Chofer	Presiona "Detener Ruta"	React <code>onClick</code>
Chofer	<code>clearWatch(watchId) + emite gps_stop {microId}</code>	Geolocation + Socket.IO
Backend	Borra Redis key + emite <code>micro_offline</code> a sala	DEL + <code>sio.emit</code>
Pasajero	Recibe <code>micro_offline</code> → elimina marcador del mapa	React state delete

8.3 Flujo de pago con tarjeta



8.4 Flujo de desvío (Chofer notifica a Pasajeros)





9. Variables de Entorno y Configuración

Archivo ubicado en `mimicro-app/.env`

Variable	Ejemplo	Descripción
DATABASE_URL	postgresql://postgres:1234@localhost:5432/mimicro_db	Conexión a PostgreSQL. Formato: usuario:contraseña@host:puerto/db
REDIS_URL	redis://localhost:6379	Conexión a Redis para caché GPS
JWT_SECRET	mimicro_clave_secreta_min32chars_2026	Clave de firma JWT. Mínimo 32 caracteres. NUNCA compartir.
JWT_EXPIRES_DAYS	7	Días de validez del token JWT
PORT	3000	Puerto del servidor backend (FastAPI)
FRONTEND_URL	http://localhost:5173	URL del frontend para CORS
LOG_FILE	security.log	Archivo JSONL de logs de seguridad

10. Código Fuente de Archivos Clave

Esta sección contiene el código fuente completo de los archivos de configuración y entrada que no se generan automáticamente — son imprescindibles para que el proyecto funcione.

backend/main.py — Punto de entrada del servidor

Instancia FastAPI, registra todos los routers, monta Socket.IO y gestiona el ciclo de vida (DB + Redis).

```
import os
from contextlib import asynccontextmanager
from dotenv import load_dotenv

load_dotenv(os.path.join(os.path.dirname(__file__), '..', '.env'))

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import socketio

from MODULO_HECTOR.config.db import init_db
from MODULO_HECTOR.config.redis_config import init_redis
from MODULO_HECTOR.config.socket_config import sio

from MODULO_BRUNO.routes.authRoutes import router as auth_router
from MODULO_BRUNO.routes.pagosRoutes import router as pagos_router
from MODULO_CRISTHIAN.routes.rutasRoutes import router as rutas_router
from MODULO_JAVIER.routes.gpsRoutes import router as gps_router
from MODULO_ROBERTO.routes.microsRoutes import router as micros_router
from MODULO_SAMUEL.routes.notifRoutes import router as notif_router
from MODULO_SAMUEL.routes.etaRoutes import router as eta_router

# Registrar handlers de Socket.IO (los eventos se registran al importar)
import MODULO_JAVIER.socket.gpsSocket # noqa: F401

@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    await init_redis()
    yield

app = FastAPI(title="miMicro API", version="1.0.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv("FRONTEND_URL", "http://localhost:5173"), "*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router, prefix="/auth", tags=["auth"])
app.include_router(pagos_router, prefix="/tarjeta", tags=["pagos"])
app.include_router(rutas_router, prefix="/rutas", tags=["rutas"])
app.include_router(gps_router, prefix="/gps", tags=["gps"])
app.include_router(micros_router, prefix="/micros", tags=["micros"])
app.include_router(notif_router, prefix="/notificaciones", tags=["notificaciones"])
app.include_router(eta_router, prefix="/eta", tags=["eta"])

@app.get("/", tags=["health"])
async def health():
    return {"status": "ok", "app": "miMicro API"}

# Montar Socket.IO sobre la app ASGI
```

```
socket_app = socketio.ASGIApp(sio, app)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(socket_app, host="0.0.0.0", port=int(os.getenv("PORT", "3000")))
```

frontend/vite.config.js — Proxy de desarrollo

Redirige todas las llamadas API relativas al backend en puerto 3000. Incluye proxy WebSocket para Socket.IO.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/auth':      { target: 'http://localhost:3000', changeOrigin: true },
      '/rutas':     { target: 'http://localhost:3000', changeOrigin: true },
      '/gps':       { target: 'http://localhost:3000', changeOrigin: true },
      '/micros':    { target: 'http://localhost:3000', changeOrigin: true },
      '/notificaciones': { target: 'http://localhost:3000', changeOrigin: true },
      '/eta':       { target: 'http://localhost:3000', changeOrigin: true },
      '/tarjeta':   { target: 'http://localhost:3000', changeOrigin: true },
      '/socket.io':  { target: 'http://localhost:3000', changeOrigin: true, ws: true },
    }
  }
})
```

frontend/package.json — Dependencias exactas

```
{
  "name": "mimicro-web",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --port 5173",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.0",
    "socket.io-client": "^4.7.5",
    "react-leaflet": "^4.2.1",
    "leaflet": "^1.9.4"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.0"
  }
}
```

backend/cpp_core/setup.py — Compilación del módulo C++

Permite compilar las clases C++ como módulo Python (`mimicro_core`) via pybind11. Se ejecuta desde `backend/cpp_core/`.

```
import os
from setuptools import setup, Extension
import pybind11

# Ruta absoluta a la raíz del repo donde están las carpetas CLASES*/
cpp_root = os.path.abspath(
```



```
os.path.join(os.path.dirname(__file__), '..', '..', '..', 'PROYECTO_ESTRUCTURA')
)

ext = Extension(
    'mimicro_core',
    sources=['bindings.cpp'],
    include_dirs=[
        pybind11.get_include(),
        os.path.join(cpp_root, 'CLASES_HECTOR'),
        os.path.join(cpp_root, 'CLASES_BRUNO'),
        os.path.join(cpp_root, 'CLASES_CRISTHIAN'),
        os.path.join(cpp_root, 'CLASES_JAVIER'),
        os.path.join(cpp_root, 'CLASES_ROBERTO'),
        os.path.join(cpp_root, 'CLASES_SAMUEL'),
    ],
    language='c++',
    extra_compile_args=['/std:c++17', '/utf-8'] if os.name == 'nt' else ['-std=c++17'],
)

setup(
    name='mimicro_core',
    version='1.0.0',
    ext_modules=[ext],
)
```

⚠ Nota sobre la integración C++

El módulo `mimicro_core` es **opcional**. Todo el backend tiene fallbacks Python puro — si el módulo no se compila, el sistema igual funciona al 100% usando listas, colas y grafos en Python nativo. Para compilar:

```
cd mimicro-app/backend/cpp_core
pip install pybind11
pip install -e .
```

Requiere compilador C++17: **MSVC** (Visual Studio Build Tools) en Windows, **GCC 9+** en Linux/Mac. Los archivos `.h` de las carpetas `CLASES */` deben estar presentes.

11. Guía Completa para Recrear el Proyecto

Paso 1 — Requisitos de software

Programa	Versión	Descarga
Git	cualquiera	https://git-scm.com/downloads
Python	3.11	https://www.python.org/downloads/release/python-3119/
PostgreSQL	14+	https://www.postgresql.org/download/
Redis	7+	https://github.com/tporadowski/redis/releases (Windows)
Node.js	18+	https://nodejs.org/en/download

Paso 2 — Clonar el repositorio

```
git clone https://github.com/HectorMarquez2705/proyecto_estructura_de_Datos.git
cd proyecto_estructura_de_Datos
```

Paso 3 — Configurar .env

```
# Editar mimicro-app/.env
DATABASE_URL=postgres://postgres:TU_CONTRASEÑA@localhost:5432/mimicro_db
REDIS_URL=redis://localhost:6379
JWT_SECRET=mimicro_clave_secreta_minimo_32_caracteres_2026
JWT_EXPIRES_DAYS=7
PORT=3000
FRONTEND_URL=http://localhost:5173
LOG_FILE=security.log
```

Paso 4 — Crear la base de datos

```
psql -U postgres -c "CREATE DATABASE mimicro_db;"
psql -U postgres -d mimicro_db -f mimicro-app/backend/database/schema.sql
psql -U postgres -d mimicro_db -f mimicro-app/backend/database/seed.sql
```

Paso 5 — Backend

```
cd mimicro-app/backend
pip install -r requirements.txt
python -m uvicorn main:app --reload --port 3000
```

Paso 5b — Compilar módulo C++ (opcional pero recomendado)

Este paso integra las clases C++ al backend vía pybind11. Si se omite, el backend usa fallbacks Python puro y funciona igualmente.

```
# Requiere Visual Studio Build Tools (Windows) o GCC 9+ (Linux/Mac)
cd mimicro-app/backend/cpp_core
pip install pybind11
```

```
pip install -e .
# El módulo mimicro_core.pyd/.so queda en backend/ y se importa automáticamente
```

Paso 6 — Frontend

```
cd mimicro-app/frontend
npm install
npm run dev
```

Paso 7 — Acceder

Abri **http://localhost:5173** en el navegador.

Cuentas de prueba del seed

Rol	Email	Contraseña
Admin	admin@mimicro.bo	Admin2026!
Chofer	pedro.rios@mimicro.bo	Chofer123!
Chofer	carlos.vaca@mimicro.bo	Chofer123!
Pasajero	juan.perez@mail.com	Pasajero123!
Pasajero	maria.lopez@mail.com	Pasajero123!

Estructura completa de carpetas generada

```
PROYECTO_ESTRUCTURA/
├─ README.md
├─ CLAUDE.md
├─ documentacion_miMicro.html      ← este documento
├─ CLASES_HECTOR/
│  └─ Analizador_Flujo_Hector.h
│  └─ Reporte_Seguridad_Hector.h
│  └─ main.cpp                    ← demo C++ standalone
├─ CLASES_BRUNO/
│  └─ persona.h
│  └─ TarjetaTransporte.h
│  └─ GestorPagos.h
├─ CLASES_CRISTHIAN/
│  └─ GrafoParadas.h
│  └─ OptimizadorRuta.h
│  └─ Ruta.h
├─ CLASES_JAVIER/
│  └─ GestorGPS.h
│  └─ Parada.h
│  └─ PilaHistorial.h
├─ CLASES_ROBERTO/
│  └─ Micro.h
│  └─ ListaPasajeros.h
│  └─ SensorPuerta.h
├─ CLASES_SAMUEL/
│  └─ ColaEspera.h
│  └─ NotificacionAlerta.h
│  └─ TiempoEstimado.h
└─ mimicro-app/
   └─ .env
   └─ backend/
      └─ main.py
```

```
├─ requirements.txt
├─ database/
│   └─ schema.sql
│       └─ seed.sql
├─ MODULO_HECTOR/
│   └─ config/db.py, redis_config.py, socket_config.py
│       └─ middleware/authMiddleware.py, rolesMiddleware.py
│           └─ utils/jwtHelper.py, encriptador.py
│               └─ services/reporteSeguridad.py
├─ MODULO_BRUNO/
│   └─ models/Persona.py, TarjetaTransporte.py
│       └─ controllers/authController.py, pagosController.py
│           └─ routes/authRoutes.py, pagosRoutes.py
├─ MODULO_CRISTHIAN/
│   └─ models/Ruta.py, GrafoParadas.py, OptimizadorRuta.py
│       └─ controllers/rutasController.py
│           └─ routes/rutasRoutes.py
├─ MODULO_JAVIER/
│   └─ models/GestorGPS.py, Parada.py, PilaHistorial.py
│       └─ controllers/gpsController.py
│           └─ routes/gpsRoutes.py
│               └─ socket/gpsSocket.py
├─ MODULO_ROBERTO/
│   └─ models/Micro.py, ListaPasajeros.py, SensorPuerta.py
│       └─ controllers/microsController.py
│           └─ routes/microsRoutes.py
├─ MODULO_SAMUEL/
│   └─ models/ColaEspera.py, NotificacionAlerta.py, TiempoEstimado.py
│       └─ controllers/notifController.py, etaController.py
│           └─ routes/notifRoutes.py, etaRoutes.py
├─ frontend/
│   └─ package.json
│       └─ vite.config.js
│           └─ index.html
│               └─ src/
│                   └─ main.jsx
│                       └─ App.jsx
│                           └─ index.css
│                               └─ context/AuthContext.jsx
│                                   └─ services/
│                                       └─ authService.js
│                                           └─ gpsService.js
│                                               └─ rutasService.js
│                                                   └─ pagosService.js
│                                                       └─ notifService.js
│                                       └─ components/
│                                           └─ Layout.jsx
│                                               └─ MapaInteractivo.jsx
│                                                   └─ Toast.jsx
│                                       └─ screens/
│                                           └─ auth/LoginScreen.jsx, RegisterScreen.jsx
│                                               └─ admin/GestionUsuariosScreen.jsx, GestionRutasScreen.jsx,
│                                                   GestionMicrosScreen.jsx, ReportesScreen.jsx
│                                               └─ chofer/RutaActivaScreen.jsx, ReportarOcupacionScreen.jsx,
│                                                   ReportarDesvioScreen.jsx
│                                               └─ pasajero/MapaScreen.jsx, ETAScreen.jsx, TarjetaScreen.jsx,
│                                                   HistorialScreen.jsx, NotificacionesScreen.jsx
```

